SUPRACODEX-HACK

Apprendre, comprendre, construire et innover.

COURS COMPLET

Séance 1 — Apprendre à raisonner comme un informaticien

Cours d'initiation — débutants acceptés

NIVEAU L1 — INITIATION À L'INFORMATIQUE

Formateur / Responsable pédagogique
JORDAN HACKER

Objectif : apprendre à analyser un problème et à écrire un algorithme simple.

Durée indicative de la séance : 2 heures

Logo provisoire — créé pour ce document, il ne représente pas un logo officiel.

1. Introduction à l'informatique

L'informatique est la science du traitement automatique de l'information (les *données*) par des machines : ordinateurs, téléphones, distributeurs de billets...

	Utiliser un ordinateur	Programmer	Faire de l'informatique
Définition	Se servir d'un logiciel déjà fait.	Écrire des instructions qu'une machine peut exécuter.	Analyser un problème, concevoir une solution, la programmer, la tester.
Exemple	Écrire un message sur WhatsApp.	Écrire le code qui calcule une moyenne.	Décider comment calculer la moyenne *avant* d'écrire le code.

Problème informatique : situation où l'on dispose de **données** et où l'on veut obtenir un **résultat précis** par un traitement automatique. *Exemple : à partir de trois notes, obtenir la moyenne et savoir si l'étudiant est admis.*

> **Pourquoi analyser avant de programmer ?** Un maçon ne construit pas sans plan. Coder sans avoir compris le problème produit des erreurs et fait perdre du temps. Un informaticien **réfléchit d'abord**, **écrit ensuite**.

2. Qu'est-ce qu'un algorithme ?

Définition. Un **algorithme** est une suite **finie** d'instructions **précises**, exécutées dans un **ordre défini**, qui permet de résoudre un problème. *Dans la vie courante : une recette de cuisine, les étapes d'un retrait au distributeur, un itinéraire pour aller à l'université.*

Un bon algorithme est :

Clair	Fini	Ordonné	Correct	Général
Chaque instruction n'a qu'un seul sens.	Il s'arrête après un nombre fini d'étapes.	L'ordre des étapes est important.	Il donne le bon résultat pour toutes les données prévues.	Il marche pour toutes les valeurs, pas pour un seul cas.

Algorithme ≠ programme. L'algorithme est la **méthode**, indépendante de tout langage. Le programme est sa **traduction** dans un langage (Python, C...) que l'ordinateur peut exécuter. **Erreur logique** : l'algorithme est écrit correctement mais sa méthode est fausse (voir page 4).

3. Le modèle Entrées → Traitement → Sorties

Tout problème se décompose en trois parties : ce qu'on **reçoit**, ce qu'on **fait**, ce qu'on **produit**.



Problème	Entrées	Traitement	Sortie
Moyenne de 2 notes	note1, note2	(note1 + note2) / 2	la moyenne
Plus grand de 2 nombres	a, b	comparer a et b	le plus grand des deux
Opération de paiement	prix, somme versée	SI versé ≥ prix : monnaie = versé − prix	la monnaie, ou « montant insuffisant »

4. Variables et types de données

Une **variable** est une zone de la mémoire, **portant un nom**, qui contient une valeur pouvant changer (comme une boîte étiquetée).

Type	Contient	Exemples
ENTIER	un nombre sans virgule	25 ; 0 ; −3
REEL	un nombre à virgule	12,5 ; 0,75
CHAINE	du texte (entre guillemets)	"Awa" ; "ADMIS"
BOOLEEN	une valeur de vérité	VRAI ; FAUX

Noms lisibles : une lettre au début, pas d'espace ni d'accent, un nom qui dit ce qu'il contient (`moyenne`, `prixTotal` plutôt que `x` ou `m2`).

Affectation et comparaison — deux choses différentes :

Notation	Sens	Exemple
`x ← 5`	**Affectation** : x reçoit la valeur 5 (c'est une action).	`a ← 4` puis `a ← a + 1` : a vaut maintenant 5.
`x = 5`	**Comparaison** : x est-il égal à 5 ? (réponse VRAI ou FAUX)	Si x vaut 5 : VRAI. Si x vaut 7 : FAUX.

Opérations arithmétiques : `+` addition, `−` soustraction, `×` multiplication, `/` division, `DIV` quotient entier, `MOD` reste. Exemple : `17 DIV 5 = 3` et `17 MOD 5 = 2`.

5. Lecture et affichage

`LIRE a` : l'utilisateur saisit une valeur, rangée dans la variable *a*. `AFFICHER ...` : le résultat est montré à l'écran.

```
LIRE a            // entrée : on reçoit une valeur
LIRE b
somme ← a + b     // traitement
AFFICHER "Somme = ", somme   // sortie
```

6. Les conditions

Une **condition** est une question dont la réponse est VRAI ou FAUX. Selon la réponse, l'algorithme choisit un chemin.

Comparaison	=	≠	<	>	≤	≥
Signification	égal	différent	inférieur	supérieur	inférieur ou égal	supérieur ou égal

```
SI a > b ALORS
    AFFICHER a, " est le plus grand"
SINON
    AFFICHER b, " est le plus grand ou égal"
FINSI
```

```
SI note ≥ 16 ALORS
    mention ← "Très bien"
SINON SI note ≥ 10 ALORS
    mention ← "Passable"
SINON
    mention ← "Insuffisant"
FINSI
```

Opérateurs logiques

A	B	A ET B	A OU B	NON A
VRAI	VRAI	VRAI	VRAI	FAUX
VRAI	FAUX	FAUX	VRAI	FAUX
FAUX	VRAI	FAUX	VRAI	VRAI
FAUX	FAUX	FAUX	FAUX	VRAI

ET : les deux conditions doivent être vraies. **OU** : au moins une est vraie. **NON** : inverse la valeur. *Exemple : (age ≥ 18) ET (note ≥ 10).*

7. Écrire en pseudo-code

Le **pseudo-code** est une façon d'écrire un algorithme en français structuré, sans les contraintes d'un vrai langage. **Convention utilisée dans toute la formation :** mots-clés en MAJUSCULES, une instruction par ligne, indentation dans les blocs SI, commentaires après //, affectation avec ←.

```
ALGORITHME nomDeLAlgorithme
VARIABLES
    a, b : ENTIER
    nom  : CHAINE
DEBUT
    LIRE a
    AFFICHER a
FIN
```

Test des valeurs

Type de test	But
Ordinaire	valeur normale, résultat connu
Limite	valeur frontière (ex. exactement 10)
Invalide	valeur interdite (ex. note 25 sur 20)

8. Tests et erreurs fréquentes

Erreur	Définition	Exemple
Syntaxe	Une règle d'écriture n'est pas respectée.	Oublier FINSI ; écrire AFICHER.
Exécution	L'algorithme s'arrête en pleine exécution.	Diviser par 0.
Logique	Tout s'exécute, mais le résultat est faux.	`n1 + n2 / 2` au lieu de `(n1 + n2) / 2`.

Table de traçage : on exécute l'algorithme à la main, **une ligne par instruction**, en notant la valeur de chaque variable après chaque étape.

9. Exemple entièrement résolu

Énoncé : lire trois notes sur 20, afficher leur moyenne et « ADMIS » si la moyenne est supérieure ou égale à 10, sinon « NON ADMIS ».

Étape	Application
1. Comprendre	Trois notes en entrée ; une moyenne et une décision en sortie.
2. Données	n1, n2, n3 (REEL), notes entre 0 et 20.
3. Résultat	moyenne = (n1 + n2 + n3) / 3 ; décision : ADMIS si moyenne ≥ 10.
4. Étapes	Lire les 3 notes → calculer la somme → calculer la moyenne → comparer à 10 → afficher.

```
ALGORITHME moyenneEtDecision
VARIABLES  n1, n2, n3, somme, moyenne : REEL
DEBUT
    LIRE n1, n2, n3
    somme   ← n1 + n2 + n3
    moyenne ← somme / 3
    AFFICHER "Moyenne = ", moyenne
    SI moyenne ≥ 10 ALORS
        AFFICHER "ADMIS"
    SINON
        AFFICHER "NON ADMIS"
    FINSI
FIN
```

5-6. Trace avec (12 ; 8 ; 13)

Instruction	n1	n2	n3	somme	moyenne	Écran
LIRE n1,n2,n3	12	8	13	–	–	
somme ← …	12	8	13	33	–	
moyenne ← …	12	8	13	33	11	
AFFICHER	12	8	13	33	11	Moyenne = 11
11 ≥ 10 ?					VRAI	ADMIS

7. Tests et correction

Test	Entrées	Résultat attendu	Résultat obtenu
Ordinaire	12 ; 8 ; 13	11 → ADMIS	11 → ADMIS ✓
Limite	10 ; 10 ; 10	10 → ADMIS	10 → ADMIS ✓
Limite (juste en dessous)	9 ; 10 ; 10	9,67 → NON ADMIS	9,67 → NON ADMIS ✓
Invalide	25 ; 10 ; 10	refuser la note 25	ADMIS ✗ (non vérifié !)

Correction : l'algorithme accepte une note de 25/20 ; il manque une vérification. On ajoutera `SI n1 < 0 OU n1 > 20 ALORS` pour refuser les notes invalides. C'est l'objet du TP.

10. Deux exercices corrigés

Exercice 1 — Pair ou impair. Lire un entier naturel et indiquer s'il est pair ou impair. *Raisonnement :* un nombre est pair si le reste de sa division par 2 est 0 ; entrée : n ; traitement : n MOD 2 ; sortie : « pair » ou « impair ».

```
ALGORITHME parite
VARIABLES n : ENTIER
DEBUT
    LIRE n
    SI n MOD 2 = 0 ALORS
        AFFICHER "pair"
    SINON
        AFFICHER "impair"
    FINSI
FIN
```

Test	n	n MOD 2	Sortie
Ordinaire	8	0	pair
Ordinaire	7	1	impair
Limite	0	0	pair

Exercice 2 — Prix d'un achat avec remise. Lire le prix unitaire et la quantité. Le total est prix × quantité. Si le total est supérieur ou égal à 100, on accorde 10 % de remise. Afficher le montant à payer. *Raisonnement :* entrées : prix, quantite ; traitement : total, puis condition sur 100 ; sortie : montant.

```
ALGORITHME achat
VARIABLES prix, quantite, total, montant : REEL
DEBUT
    LIRE prix, quantite
    total ← prix × quantite
    SI total ≥ 100 ALORS
        montant ← total − total × 0,10
    SINON
        montant ← total
    FINSI
    AFFICHER "À payer : ", montant
FIN
```

Test	prix ; qté	total	À payer
Ordinaire	40 ; 3	120	108
Limite	50 ; 2	100	90
Sans remise	30 ; 2	60	60

11. Fiche de révision

- **Algorithme** = suite finie d'instructions précises et ordonnées qui résout un problème.
- Toujours : **Entrées → Traitement → Sorties**. Comprendre avant de coder.
- **Variable** = boîte nommée ; ← affecte, = compare.
- **SI … SINON … FINSI** : choisir selon une condition (VRAI/FAUX) ; ET, OU, NON les combinent.
- **Méthode :** comprendre → données → résultat → étapes → pseudo-code → tester → corriger.
- **Toujours tester :** valeur ordinaire, valeur limite, valeur invalide.

12. Mini-glossaire

Algorithme	Suite finie d'instructions qui résout un problème.
Programme	Algorithme traduit dans un langage exécutable.
Donnée	Information fournie ou calculée.
Variable	Zone mémoire nommée contenant une valeur.
Affectation	Action qui donne une valeur à une variable (←).

Condition	Question dont la réponse est VRAI ou FAUX.
Booléen	Type à deux valeurs : VRAI ou FAUX.
Trace	Exécution à la main, étape par étape.
Test limite	Test sur une valeur frontière (ex. 10 pour « ≥ 10 »).
Erreur logique	Algorithme qui s'exécute mais donne un faux résultat.